

# Reproducing GoSPA: Implementation of a Sparse CNN Accelerator

Sara Ahmad, Mahdi Hassen, Fred Huang, Adil Kazimov, Emon Sarkar

Dept. of Electrical and Computer Engineering, University of Waterloo, Waterloo, Canada  
{s92ahmad, mhassen, j462huan, akazimov, esarkar}@uwaterloo.ca

**Abstract**—Convolutional Neural Networks (CNNs) exhibit high sparsity: after pruning and ReLU, 70-90% of weights and activations are zero, yet dense hardware accelerators still spend clock cycles and energy on zero-valued multiplications. This work reproduces GoSPA, a sparse CNN accelerator that exploits unstructured sparsity through on-the-fly intersection: each nonzero activation is tagged with a position ID (PID) and a convolution ID (CID), which are used to implicitly intersect weight and activation indices and identify only the weight-activation pairs that perform useful computation. We implement the full accelerator, including the Activation Processing Unit (APU) and the Processing Element (PE) array, together with performance models that we cross-verify against the RTL. Our final Kria KV260 instances close timing at 274.1 MHz for a pruned AlexNet configuration and 250.1 MHz for a quantized MobileNetV2 configuration, reaching 49.6 fps on AlexNet and 56.1 fps on MobileNetV2.

**Index Terms**—sparse convolution, CNN accelerator, unstructured sparsity, FPGA, SystemVerilog, hardware accelerator

## I. INTRODUCTION

### A. Motivation

Convolutional neural networks (CNNs) have become a dominant computational workload, but the accuracy gains of successive models have come from ever-larger networks whose inference increasingly strains the hardware that runs them. Much of that computation, however, is wasted effort: a large fraction of the operands are zero. Weight pruning deliberately zeros out redundant connections, and the ubiquitous ReLU activation clamps every negative value to zero, resulting in 70-90% of both the weights and the activations vanishing in a typical layer. Any multiplication involving one of these zeros produces nothing, yet a dense GPU or systolic array still spends clock cycles and energy computing them. Skipping this ineffectual work promises a large improvement in both throughput and energy.

Exploiting the sparse pattern is a difficult problem. When sparsity is structured, its nonzeros restricted to a predictable pattern, the hardware knows in advance where the useful operands lie. Pruned CNNs, however, are unstructured, i.e., the nonzeros are scattered irregularly, and the accelerator must discover them at runtime. That discovery is never free: the extra machinery a design introduces to locate the useful multiplications - to index, match, and route the sparse operands; carries its own cost in silicon area, memory and interconnect resources, latency, and energy, and can even add redundant computation of its own. Depending on the mechanisms a

design adopts, these overheads can partially or entirely offset the savings that motivated exploiting sparsity in the first place.

### B. GoSPA

Prior works, such as SparTen [2] and ExTensor [3], commonly handle such unstructured sparsity with explicit intersection where cycles must be spent before and in between the actual computation to identify the zero activations and weights, as well as associating their corresponding positions in a convolution operation such that those zero multiplications can be skipped. This leads to three major drawbacks [1]: (1) Energy and silicon cost of the intersection operation are typically high; (2) Overhead: the data dependency between the intersection logic and the multiplication processing elements (PEs) implies that the PEs will have to wait for the intersection results to start computing the output activations; (3) Unnecessary data transfer, where the zero values will need to be loaded from SRAM for the intersection logic, even if those values will be excluded from the compute.

GoSPA [1] targets 2-D unstructured sparse convolution and structurally avoids the drawbacks above. A key observation that motivates GoSPA is that, for a given CNN, the sparse weights can be seen as a static stream that does not require runtime handling. Since the weights are known prior to inference, it is possible to pre-load the knowledge about weight sparsity, and use that to intersect with the dynamic activations within the PEs (on-the-fly intersection [1]), rather than through explicit intersection operations that must take place before the computation.

### C. Contribution

This project reproduces and analyzes GoSPA. Our contributions are:

- Python functional and performance models that guide the RTL design and are cross-verified against it.
- A parameterized SystemVerilog implementation of the full accelerator, APU Stage 1 (ID generation), Stage 2 (routing), and the PE array.
- Two implemented interpretations of the paper's multi-lane dataflow - multiple kernels per PE (V1) and one kernel per PE (V2) - used as a design-space exploration to identify the throughput-bounding stage and guide the final architecture.

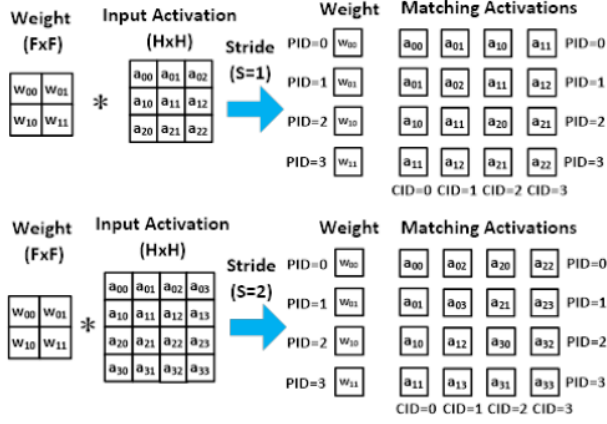


Fig. 1. PID and CID assignment in GoSPA's compute reordering, taken from [1].

- GoSPA V3: a single, runtime-stride-configurable instance built on V2's dataflow, with output tiling and a controller module.
- Workload Testing: Testing GoSPA versions with AlexNet and MobileNetV2 networks.

## II. BACKGROUND: EXPLOITING SPARSITY AND GOSPA DATAFLOW

GoSPA achieves higher utilization and lower latency compared to prior sparse accelerators that consider weights as dynamic streams unknown at runtime and thus require costly units to search streams and find matches at runtime. GoSPA recognizes that for DNN inferences, the weights are known at runtime, and therefore a more efficient solution is possible. With a static weight stream and a general dynamic activation stream, GoSPA uses on-the-fly intersection and computation reordering to perform only necessary activations in an efficient manner.

### A. Compute Re-ordering

GoSPA's on-the-fly intersection relies on a specialized compute re-ordering mechanism. Each input activation is tagged with two IDs: a Position ID (PID) that indicates the indices of the weights in the 2-D convolution kernel that this activation will multiply with; and a Convolution ID (CID) which indicates the indices of the output pixel that this activation will affect. This mechanism is shown in Figure 1 [1]. After the zero activations are filtered out from the input stream, the main job of a PID is to decide whether the tagged activation should be sent into a PE for multiplication based on the weight value at the position. The output of the multiplication will be accumulated based on the activation's CID into the corresponding output pixel.

Weight sparsity is encoded within the PEs by a specialized data structure called Weight Sparsity Pattern (WSP). It is a bit mask of length  $F^2$ , where  $F$  is the dimension of a 2-D convolution kernel. The  $n$ -th bit indicates whether the  $n$ -th weight, in row-major order, is zero. In the upper part of Figure

1, for example, if all  $w_{00}, w_{01}, w_{02}$  are 0 while  $w_{03}$  is nonzero, then the WSP for this kernel will be 0001, and subsequently, only the activations with PID=3 will be processed by a PE.

Since the PID/CID assignment follows directly from the convolution geometry  $(F, H, S)$ , and the WSP value of the kernel is known in advance, the intersection lookup between an Activation's PID and a PE's corresponding WSP bit is a simple logical operation.

### B. Activation Processing Unit (APU)

The GoSPA hardware architecture is designed largely in accordance with this dataflow. Figure 2 [1] illustrates the structure from a high level: an Activation Processing Unit (APU) feeds an array of Processing Elements (PEs). The APU converts the raw sparse activation stream into PID/CID-tagged work items and delivers each item only to the PEs that will use it. It consists of an activation SRAM and two pipeline stages. The SRAM holds several rows of the current input channel in compressed sparse row (CSR) form and operates as a circular buffer: an incoming row replaces the oldest resident row. Restricting processing to a window of rows bounds the CID range, keeping the ID fields narrow.

**Stage 1: ID generation.** A pre-processing block converts each nonzero activation's CSR metadata into its  $(x, y)$  coordinates. An array of IDGen units, each containing comparators and MAC units, then enumerates in parallel every kernel position that overlaps the activation, producing all valid (PID, CID) pairs after a boundary check. The tagged activations are pushed into a bank of  $N_{PID} = F^2$  FIFO-A queues, one per PID; the PID itself is implicit in the bank index and never stored.

**Stage 2: WSP-gated routing.** The routing module selects one (activation, CID) pair per cycle from the FIFO-A banks and broadcasts it toward all  $N_{PE}$  FIFO-B queues, each feeding one PE. A per-PE switch gates the broadcast: the entry is written into FIFO-B  $k$  only if  $WSP_k[PID] = 1$ , i.e., only if PE  $k$  actually holds a nonzero weight at that kernel position. Activations paired exclusively with zero weights are therefore dropped in flight—the on-the-fly intersection costs one AND gate per PE rather than a dedicated intersection unit, and no zero operand is ever transferred to a PE.

### C. Processing Element (PE) Array

Each PE stores the nonzero weights of its assigned filter kernel(s) in a local SRAM, ordered by PID, together with the kernel's WSP. Two registers, *Curr\_Wgt* and *Next\_Wgt*, hold the current and next nonzero weight and their PIDs. When a tagged activation arrives from FIFO-B, its PID is compared against *Curr\_Wgt*'s PID: on a match the pair is multiplied immediately; otherwise the registers slide forward (*Next* becomes *Curr*) and a new weight is fetched from SRAM. Because Stage 2 drains FIFO-A banks in PID order, equal PIDs arrive in adjacent cycles, so each weight stays resident in the register for its maximum useful lifetime such that we maximize register-level reuse and minimize SRAM reads. Each product is accumulated into an accumulator register

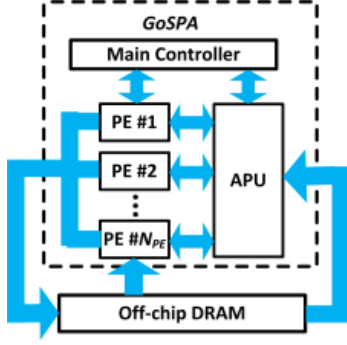


Fig. 2. High level hardware architecture of GoSPA, taken from [1].

file indexed by the activation’s CID; when a kernel’s stream completes, the accumulators hold the output pixels, which pass through ReLU and are written back in CSR form. Since ID matching is deterministic arithmetic rather than a runtime search, the APU can keep every PE supplied each cycle, and the whole path is pipelined with no intersection stalls on the critical path.

### III. RTL IMPLEMENTATION DETAILS

The accelerator is written in parameterized SystemVerilog and organized as a top module (`gospa`) wrapping the APU and the PE array. The host-facing interface exposes five signal groups: CSR activation fill, WSP load, scan/route framing, PE weight load, and accumulator drain. Every architectural dimension is a synthesis-time parameter, which is what enables the design-space sweeps of Section VI. Table I lists the principal parameters and the configuration used for the reported results; the throughput knobs in the last group default to 1 and are the subject of the optimization study in Section VI.

TABLE I  
DESIGN PARAMETERS (REPORTED CONFIGURATION).

| Parameter       | Meaning                              | Value |
|-----------------|--------------------------------------|-------|
| $H$             | activation map size ( $H \times H$ ) | 8     |
| $F$             | kernel size ( $F \times F$ )         | 3     |
| $S$             | stride                               | 1     |
| $N_{PE}$        | PEs / output channels in parallel    | 8     |
| $N_{MULTS}$     | multiplier lanes per PE              | 4     |
| $FIFO\_D$       | FIFO-A / FIFO-B depth                | 64    |
| $DATA\_W$       | activation / weight width            | 16    |
| $ACC\_W$        | accumulator width                    | 32    |
| $N_{PID}$       | FIFO-A banks ( $= F^2$ )             | 9     |
| $N_{CID}$       | accumulators/lane ( $= E^2$ )        | 36    |
| $LAT$           | multiplier latency (cycles)          | 6     |
| $FILL\_W$       | CSR entries filled per cycle         | 1     |
| $STAGE1\_BATCH$ | activations enumerated per cycle     | 1     |
| $DRAIN\_W$      | accumulators drained per PE/cycle    | 1     |

#### A. APU Stage 1

Stage 1 turns the raw activation stream into PID-tagged entries. The activation feature map is held on-chip in compressed sparse row (CSR) form, so only nonzero activations are stored

and zeros are skipped without any dedicated logic. The store is split into an entry SRAM, holding  $\{\text{value}, \text{column}\}$  pairs in scan order, and a row-pointer table held in flops; the host fills both through a  $FILL\_W$ -wide port. On `scan_start`, a scanner walks the row-pointer table and streams entries out of the SRAM; a decode block (`csr_decode`) recovers each entry’s  $(x, y)$  coordinate, and the ID-generation block (`idgen, position_encode`) enumerates every kernel position that overlaps that coordinate, emitting all boundary-valid (PID, CID) pairs. Up to  $STAGE1\_BATCH$  activations are enumerated per cycle. Each tagged tuple is pushed into a bank of  $N_{PID} = F^2$  FIFO-A queues indexed by PID, so the PID itself is never stored.

#### B. APU Stage 2

Stage 2 drains the FIFO-A banks in ascending PID order and packs the entries into *beats*: up to  $N_{MULTS}$  activations that share one PID but carry distinct CIDs, with a per-lane valid mask covering short beats. A show-ahead FIFO organization lets the router read  $N_{MULTS}$  entries from the active bank in a single cycle, so a full-width beat can be formed every cycle. Each beat is broadcast toward all  $N_{PE}$  FIFO-B queues and gated per PE by that PE’s WSP bit for the current PID, implementing the on-the-fly intersection. The multicast is all-or-nothing: the FIFO-A pop is held until every WSP-selected PE can accept the beat, which keeps the streams PID-monotone in every FIFO-B. The router can emit up to  $S2\_BEATS$  beats per cycle, and the FIFO-B depth ( $FIFOB\_D$ ) decouples the router from transient PE stalls.

#### C. PE

Each PE holds *one* filter kernel (V2 mapping, Section III-D) and computes one output channel per pass. It is built from three blocks. `pe_mem` stores the kernel’s nonzero weights in a PID-sorted bank, tracks the kernel’s WSP, and exports it to the Stage-2 router; the bank is double-buffered so the next round’s kernel can be loaded while the current one computes. `pe_fetch` maintains a Curr/Next weight window: the incoming beat PID is compared against the resident weights, and the matching weight is selected and broadcast to all lanes—since Stage 2 delivers PID-monotone streams, the window only ever slides forward, and each weight is fetched from the bank exactly once per pass. Finally,  $N_{MULTS}$  `pe_lane` instances each multiply their own activation from the beat by the shared weight and accumulate into a private CID-indexed accumulator bank; giving every lane its own bank provides  $N_{MULTS}$  conflict-free writes per cycle. On `drain_start`, the PE streams its accumulators out at  $DRAIN\_W$  CIDs per cycle, summing the per-lane banks for each CID; the drain is held off until every lane’s multiplier pipeline has flushed.

#### D. Design Iterations

Module boundaries stay largely the same across iterations; we built three versions of the dataflow to resolve the paper’s underspecified multi-lane PE mapping.

1) *Version 1. Multiple Kernels per PE*: Each PE holds  $N\_MULTS$  kernels, one per lane, and Stage 2 emits one activation per beat, broadcast to all lanes and gated on the *union* of their WSPs. This is the natural scale-up of the paper’s single-multiplier PE, but union gating wastes non-matching lanes, and the one-activation-per-cycle router caps the array below its multiplier count – Stage 2, not the PEs, is the bottleneck.

2) *Version 2. One Kernel per PE*: V2 moves parallelism from weights to activations: one kernel per PE, with Stage 2 packing  $N\_MULTS$  same-PID activations into each beat so the single selected weight feeds all lanes. Every admitted beat now does up to  $N\_MULTS$  useful MACs, WSP gating is exact rather than a union, and throughput scales linearly with  $N_{PE}$ .

3) *Version 3. One Kernel per PE + Tiled Output, Configurable Strides*: V3 keeps V2’s one-kernel-per-PE dataflow and adds output tiling, a runtime-configurable stride, and a controller module, so a single instance runs any layer of a real network end to end rather than requiring several instances.

#### E. V3 Design Changes

In an attempt to match the results reported in the paper, V2 was optimized and tagged GoSPA V3. The PE dataflow stayed intact, so the architecture still performs  $1 \text{ weight} \times 4 \text{ activations per PE}$ . The modification was APU’s input, it was batched so that activation consumption stayed balanced end to end. The main changes to the APU were:

- The APU SRAM was adjusted to store more activations
- The ID-generation arrays were replicated  $STAGE1\_BATCH$  times.
- The FIFO-A logic was significantly rewritten to accept  $STAGE1\_BATCH$  different activations while maintaining correct ordering.
- Output tile size was swept to find a favorable tradeoff between performance and area, and the best configuration applied.

Since AlexNet layers use different kernel sizes, we also partitioned larger kernels, such as  $5 \times 5$ , into the  $3 \times 3$  blocks that our GoSPA natively supports.

Another significant change was the introduction of a controller module. Previously, control was embedded in the testbench, which hid it from the area report and overcomplicated the host interface. The controller module offloads internal FSM sequencing from the host, leaving a simple host interface.

Table II lists the new elaboration-time parameters V3 introduces on top of Table I, together with their value in a reported AlexNet configuration.

TABLE II  
NEW PARAMETERS INTRODUCED IN V3.

| Parameter    | Meaning                           | Value |
|--------------|-----------------------------------|-------|
| E            | output tile size ( $E \times E$ ) | 16    |
| CH_B         | passes batched per tile           | 4     |
| N_ROWS       | act. ring-SRAM depth (rows)       | 63    |
| N_NZ_MAX     | act. ring-SRAM depth (nonzeros)   | 16384 |
| FIFOA_GROUPS | FIFO-A bank partitions            | 4     |
| N_SLOT       | router broadcast slots per PE     | 8     |

## IV. TESTING METHODOLOGY

Verification proceeds bottom-up. Every leaf module (FIFO, SRAM, the pipelined multiplier and multiply-add units, Stage-1 ID generation, Stage-2 routing, and the PE) has its own testbench under `testing/`, driven by Python through Verilator co-simulation. At the system level, a host-model driver executes the full operating sequence against the `gospa` top: it loads weights and WSPs, streams the CSR-encoded activation into the APU, pulses the scan/route/drain controls, and collects the per-PE output maps.

Correctness is checked at two levels. At the output level, a PyTorch model provides the reference feature map for every layer, and every output channel of every RTL run must match it exactly. Below that, the Python functional model of Section V mirrors the accelerator’s internal state as the dataflow executes—the contents of each FIFO-A bank, each per-PE FIFO-B stream, and each CID-indexed accumulator file—so a failing run can be localized to the first structure whose contents diverge from the model, instead of being observed only as a wrong output. The real-workload tests extract layer tensors from a quantized MobileNetV2 running on a real  $80 \times 80$  image; all 52 layers of the network are executed through the RTL and checked against the reference, including multi-input-channel accumulation with mid-stream weight swaps, tiled execution for memory-constrained configurations, and AlexNet layers as a second workload.

The same harness doubles as the performance-measurement infrastructure. Instrumented simulation runs report cycles, useful MACs, multiplier utilization, and a per-phase cycle attribution (fill, scan, routing, weight swap, drain). Sweep drivers automate the design-space studies reported in Section VI: an activation/weight density sweep, FIFO-depth sweeps, an  $N_{PE} \times M$  architecture sweep, and width sweeps of the  $S2\_BEATS$ ,  $STAGE1\_BATCH$ , and  $DRAIN\_W$  knobs. Except where a number is explicitly labeled as modeled, every performance figure in this paper is RTL-measured on a golden-checked run.

All three architecture versions are maintained side by side in the repository—`rtl/V1`, `rtl/V2`, and `rtl/V3_Final`—each with its own passing testbench tree under `testing/` (the V1 tree vendors the functional-model version it was verified against), and every figure and table in this paper regenerates from committed test and plotting scripts. An artifact runner (`testing/V2/artifact/run.sh`) reproduces the V1/V2 RTL measurements—including the matched comparison of Table IV—with one command from a clean checkout.

This V1/V2 testbench flow, however, verified each network layer against its own separately instantiated, elaboration-time-tuned copy of GoSPA—workable for the per-layer sweeps of Section VI, but neither representative of the paper’s single-instance setup nor practical to verify end to end on a full network. Closing that gap required the single, runtime-configurable instance verified next.

### A. V3 Verification

To reduce area and produce results comparable to the original GoSPA, the design was switched to a single instance tailored to a specific model. This approach introduced a complication: since the ID-generation array computes PIDs from the stride, and stride varies across convolution layers, the stride could no longer remain an elaboration-time constant and instead had to become a runtime parameter. The host now supplies the stride associated with a given layer to GoSPA, allowing it to adjust its configuration dynamically.

An attempt to run both AlexNet and MobileNet on a single shared GoSPA instance resulted in significant performance degradation on MobileNet, owing to its pointwise and depthwise convolution layers. A separate elaboration-time configuration was therefore created for MobileNet, yielding substantial area savings and improved performance.

To accommodate changes in our verification approach, a new `gospa_compiler` was written from scratch. It is a scheduling layer that lowers an arbitrary convolution layer onto the fixed GoSPA instance by tiling the input into  $H \times H$  windows and grouping output channels into passes.

## V. WORKLOADS AND SOFTWARE SIMULATOR

Alongside the RTL and its testbenches, a large portion of our design flow relied on two pure-software models: a functional model and a performance model. The performance model is a per-stage estimator and not a cycle accurate simulator. These simulators informed our RTL architecture by providing insights on the flow’s bottleneck and helping us iterate to our second version. Hardware parameters including, number of PEs, number of multipliers across channels, architecture version, frequency, and FIFO depths, are configured instead of hardcoded.

### A. Functional Model

The functional model works as a golden reference and implements the GoSPA dataflow in Python. It includes the CSR nonzero decoding, CID and PID generation, FIFO-A routing, WSP-gated Stage-2 routing, per-PE FIFO-B streams, multiplication, and CID-indexed accumulation stages. The output was used to verify the RTL’s accuracy by comparing a conventional dense convolution in Python against the RTL’s output. It also provides the host-side utilities the testbenches use (CSR encoding, WSP union computation, and the V2 weight chunking).

Similar to the RTL design, the functional model has two versions of the PE: “channel,” with  $M$  output-channel kernels per PE, and “act,” with a single kernel per PE shared by all  $M$  multipliers. The analytical performance model reuses the functional model’s routing and PE modules directly rather than duplicating the dataflow logic.

### B. Analytical Model

The analytical model estimates performance without simulating RTL. For each pass it replays the functional model’s golden dataflow to recover the real routed activation and

weight streams, then charges every stage the time it would need to consume them at its designed per-cycle rate. Cycle counts are built bottom-up through four nested levels, each aggregating the one below:

- The **PE** level counts, for a single PE, the cycles it spends active on its routed FIFO-B stream plus the cycles it stalls reloading weights.
- The **pass** level compares four candidate costs – Stage-1 (ID generation), Stage-2 (routing), the slowest PE, and memory – and takes the largest as both the pass’s cycle count and its bottleneck.
- The **layer** level sums passes over the output-channel tiling and input-channel accumulation loops, charging pipeline fill and drain once.
- The **network** level sums layers into total cycles, latency, and FPS, and reports speedup against a dense baseline at the same configuration.

All four share one configuration object holding  $N_{PE}$ , multipliers per PE, Stage-2 batch size, memory bandwidth, and clock frequency.

Because stage costs come from designed rates and inter-stage FIFO back-pressure is not modeled, the result is a throughput estimator rather than a cycle-accurate simulator. Its value is comparative: it ranks configurations and names the bottleneck stage. That is what drove the architecture – Stage 2, not the PE array, bounded V1, which motivated V2’s dataflow and the batched routing of Section III. The model also reports  $B^*$ , the narrowest Stage-2 batch width at which Stage 2 stops being the bottleneck.

Beyond cycles and FPS, three metrics carry most of the diagnostic weight: array utilization, which charges every idled multiplier slot rather than only the bottleneck PE’s; the load-imbalance ratio between the slowest PE and the mean; and a per-PE split of lane utilization into *overall* (useful MACs over active+stall cycles) and *compute-only* (over active cycles alone), whose gap separates the reload penalty from union under-utilization.

### C. Workloads

Workloads are run through a software path, driven purely by the performance models, and an RTL path, driven through the cocotb testbenches of Section IV. Both draw on the same network definitions and the same golden functional model, so a workload’s software result and its RTL result are comparable.

1) *Software-Only Workloads*: AlexNet and MobileNetV2 are defined as a flat list of layer descriptors and run through the analytical model of Section V. Early runs use placeholder densities to explore the router-width design space (Table III, Fig. 3); later runs use per-layer densities measured from the real apple-image tensors to compare the V1 and V2 dataflows, which is what first exposed V1’s Stage-2 bottleneck.

2) *RTL-Verified Workloads*: Every RTL workload golden-checks against the same functional model, so a passing cocotb run is a verified result; V1 and V2 each have their own independently passing testbench tree. Synthetic workloads, driven by random Bernoulli-sparse activations and kernels,

cover base correctness, density and scheduling sweeps, and per-phase cycle attribution.

Real-tensor workloads reuse activation and weight tensors captured once from pretrained, integer-quantized AlexNet and MobileNetV2 run on a real photographed apple image. MobileNetV2 runs both as its first layer alone and as its full 52-layer stack; AlexNet runs as its five real convolution layers, with a separate router/PE load-imbalance study on the first. A final workload maps that same first layer through the tiling compiler onto a deliberately undersized array, confirming that tiling costs only bounded overhead and no loss of correctness.

Together these workloads span a synthetic correctness baseline, sensitivity sweeps, and the fully pretrained networks reported end to end in Section VI.

## VI. RESULTS

### A. Performance Model Results

We ran the analytical model on AlexNet and MobileNetV2 with  $N_{PE} = 8$ ,  $M = 4$ ,  $FREQ = 1$  GHz, the “channel” dataflow, and synthetic sparsity ( $d_a = 0.5$ ,  $d_w = 0.3$ ) as a phase-1 stand-in for measured activation/weight sparsity. Table III summarizes the network-level output.

TABLE III  
ANALYTICAL-MODEL OUTPUT FOR ALEXNET AND MOBILENETV2  
( $d_a = 0.5$ ,  $d_w = 0.3$ ,  $N_{PE} = 8$ ,  $M = 4$ ).

| Network     | Cycles | Lat. (ms) | FPS   | Util. | Speedup      |
|-------------|--------|-----------|-------|-------|--------------|
| AlexNet     | 20.15M | 20.15     | 49.6  | 26.6% | $2.08\times$ |
| MobileNetV2 | 6.64M  | 6.64      | 150.5 | 28.2% | $1.76\times$ |

As we analyzed the utilization of the V1 and V2 architectures, we discovered our utilization fell significantly below the original paper’s near-full utilization. Therefore, we used our performance model to control the hardware configurations (e.g., multipliers per lane, number of PEs, batch size) and evaluate the resulting performance changes. This model simulates the dataflow and gathers various performance metrics. Through the performance model, we revealed that batch size was a key factor to boost PE/multiplier utilization: with the default router width of  $B = 1$ , the PEs are heavily stalled, while widening the router to  $B = 4$  already lifts utilization close to the density-limited ceiling, as shown in Fig. 3. Widening the router further, to  $B = 6, 8$ , or  $12$ , yields negligible additional gain over  $B = 4$  at every density tested – Stage-2 batching saturates quickly, and beyond  $B = 4$  utilization is governed almost entirely by the weight/activation density  $d_w/d_a$  rather than by router width.

### B. Design-Space Exploration: V1 vs V2

V1 and V2 architectures were synthesized out-of-context in Vivado 2024.2 for the Xilinx Kria KV260 (`xck26-sfvc784-2LV-c`) in the configuration of Table I, and both were constrained to the same aggressive 1 GHz target so that the tool reports the true achievable frequency rather than stopping at a loose constraint. Both instantiate the same custom pipelined multiplier, so the comparison isolates the

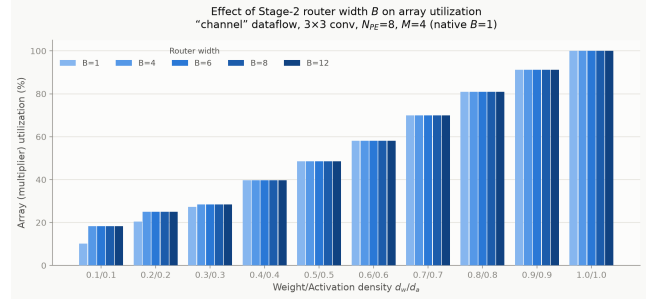


Fig. 3. Effect of Stage-2 router width  $B$  on array utilization, “channel” dataflow,  $3 \times 3$  conv,  $N_{PE} = 8$ ,  $M = 4$  (native  $B = 1$ ).

dataflow rather than the arithmetic implementation. Table IV reports the post-implementation results. Two properties hold across both versions. First, the design uses zero DSP blocks: the custom pipelined arithmetic units replace the multiplier operator, so no hard multiplier is inferred. Second, it uses zero block RAMs; at these depths the FIFOs, activation SRAM and accumulator banks are realized in the fabric, predominantly as flip-flops, with LUTs configured as distributed RAM.

TABLE IV  
FPGA RESOURCE UTILIZATION AND TIMING FOR THE TWO DESIGN VERSIONS (KRIA KV260, POST-IMPLEMENTATION, IDENTICAL 1 GHz TARGET AND IDENTICAL ARRAY CONFIGURATION). AVAILABLE: 117,120 LUT, 234,240 FF, 1,248 DSP, 144 BRAM.

|           | V1             | V2             |
|-----------|----------------|----------------|
| $F_{max}$ | 212 MHz        | 171 MHz        |
| LUT       | 36,349 (31.0%) | 42,698 (36.5%) |
| FF        | 59,952 (25.6%) | 72,610 (31.0%) |
| DSP       | 0 (0%)         | 0 (0%)         |
| BRAM      | 0 (0%)         | 0 (0%)         |

1) *Resource Utilization Sweeps*: Figures 4 and 5 report LUT and FF utilization for V1 and V2 as each architecture’s three design parameters are swept independently, holding the other two fixed.

**PE count.** Both architectures scale similarly with PE count: LUT and FF usage grow non-linearly from 2 to 16 PEs, with FF consistently above LUT and the gap widening at the high end of PE count. Since this trend is essentially the same for both designs, PE replication cost is not what distinguishes V1 from V2; the difference lies in the other two parameters.

**Per-PE parallelism.** V1’s second sweep varies kernels-per-PE ( $M$ ), the number of independent kernels each PE can hold. V2 has no directly analogous sweep; instead, its second parameter, batch width ( $B$ ), controls how many input batches are processed concurrently. V1 supports only a single input batch, so no equivalent sweep exists for it. V2’s batch-width sweep is the steepest curve in either figure: LUT usage climbs nearly  $10\times$  across  $B = 1 \rightarrow 8$ , nearly closing the gap with FF ( $\sim 130k$ ) by  $B = 8$ . Every other sweep in both figures keeps FF a consistent  $1.5\text{--}2\times$  above LUT, so batch parallelism appears to cost LUTs disproportionately – consistent with each additional batch lane duplicating combinational kernel

logic (adders/muxes) rather than adding proportionally more pipeline registers.

**FIFO depth.** The clearest architectural divergence is the FIFO depth sweep. For V1, resource usage is essentially flat across depths 16–128 ( $\sim 35k$  LUT,  $\sim 60k$  FF), meaning FIFO depth is nearly free – consistent with the zero-BRAM result above, since V1’s single-batch FIFOs are realized entirely in fabric as distributed-RAM shift registers rather than block memory, and their LUT cost does not scale meaningfully with depth over this range.

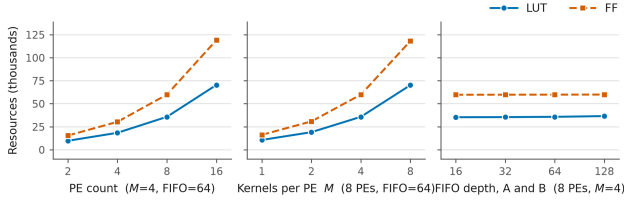


Fig. 4. V1 resource sweep

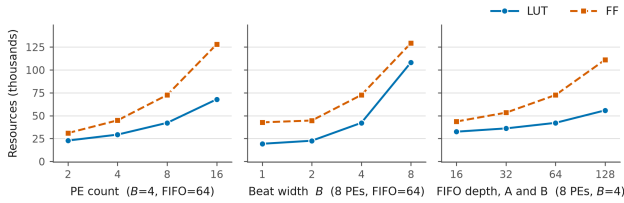


Fig. 5. V2 resource sweep

### C. GoSPA V3 Area and Timing

The parameter cost trends established by the preceding sweeps informed a further round of optimization, from which the two V3 configurations reported here were derived. Table V reports post-route resource utilization and timing for both V3 configurations on the Kria KV260. LUT usage sits at just 31% of the device for AlexNet (8 PEs, 32 multipliers) and 26% for MobileNet (16 PEs, 64 multipliers), with FF, DSP, and BRAM usage even lower, so a large fraction of the fabric is unused at either config size. Despite its larger multiplier array, the MobileNet instance’s LUT and FF usage both sit *below* the AlexNet instance in absolute terms, and its BRAM usage is a fifth of AlexNet’s – the result of deliberately optimizing the MobileNet configuration for area so that more multipliers could be packed in without exceeding AlexNet’s footprint (Section VI-D). Both instances close timing against a 250 MHz target – AlexNet with margin to spare ( $F_{\max} = 274.1$  MHz), MobileNet only barely – so every throughput number in the next section is quoted at 250 MHz, a rate both configurations sustain post-route.

TABLE V  
POST-ROUTE FPGA RESOURCE UTILIZATION AND TIMING, V3 ALEXNET (8 PEs, 32 MULTIPLIERS) VS. MOBILENETV2 (16 PEs, 64 MULTIPLIERS) CONFIGURATIONS (KRIA KV260).

|             | AlexNet         | MobileNet       |
|-------------|-----------------|-----------------|
| Target Freq | 250.0 MHz       | 250.0 MHz       |
| $F_{\max}$  | 274.1 MHz       | 250.1 MHz       |
| Slack (WNS) | +0.052 ns       | +0.001 ns       |
| LUT         | 36,327 (31.02%) | 30,384 (25.94%) |
| FF          | 18,958 (8.09%)  | 14,694 (6.27%)  |
| DSP         | 32 (2.56%)      | 32 (2.56%)      |
| BRAM        | 40 (27.78%)     | 8 (5.56%)       |

### D. Network-Level Results (V3)

To test V3, AlexNet and MobileNet are both pruned with PyTorch to a weight-sparsity ratio of 63%, matching the paper’s values. Table VI compares each layer’s measured result against the theoretical ceiling of the same configuration that is running at 100% multiplier utilization.

TABLE VI  
THEORETICAL PEAK VS. TESTBENCH MEASURED ALEXNET PERFORMANCE ( $N_{PE}=8$ ,  $N_{MULTS}=4$ , 250 MHz, 8 GMAC/S PEAK, 63% WEIGHT SPARSITY).

| Layer   | MACs (M) | Theo. FPS | Theo. Util | RTL FPS | RTL Util |
|---------|----------|-----------|------------|---------|----------|
| conv1   | 25.65    | 311.9     | 100%       | 167     | 60.5%    |
| conv2   | 40.38    | 198.1     | 100%       | 127     | 65.3%    |
| conv3   | 20.34    | 393.3     | 100%       | 284     | 73.5%    |
| conv4   | 7.17     | 1115.4    | 100%       | 640     | 60.4%    |
| conv5   | 5.38     | 1486.0    | 100%       | 836     | 59.0%    |
| Network | 98.93    | 80.9      | 100%       | 49.6    | 64.7%    |

The network consequently attains 49.6 fps, corresponding to 61% of the configuration’s theoretical peak of 80.9 fps (Table VI). This represents a strong result, as the theoretical peak assumes idealized conditions – 100% multiplier utilization with no cycles lost to weight-reload, activation-load, or drain overhead. Multiplier utilization of approximately 85% is achievable at the cost of increased area, by raising `STAGE1_BATCH` from 8 to 16. The low multiplier utilization in Table VI is attributable to router starvation, which stalls the PEs. Our best configuration reached approximately 65.9 fps at 86.9% utilization, at an area footprint of 80%. Since the GoSPA paper advocates for area savings, a design with a more modest area footprint was selected instead.

Table VII repeats this comparison for MobileNetV2, on the larger MobileNet-tuned array ( $N_{PE}=16$ , 64 multipliers, 16 GMAC/s peak – twice AlexNet’s array, so rows are not directly comparable to Table VI). MobileNetV2’s 52 layers split sharply into two populations: the  $1 \times 1$  pointwise convolutions (expand and project) keep the array fed and land in the 70-93% multUtil range, while the  $3 \times 3$  depthwise convolutions – one multiply per PID lane with no output-channel reuse to hide router latency – stall at 9-20%. We show four representative layers (conv1 through conv4) rather than all 52 rows to save space.

TABLE VII

THEORETICAL PEAK VS. RTL-MEASURED MOBILENETV2 PERFORMANCE  
( $N_{PE}=16$ ,  $N_{MULTS}=4$ , 64 MULTIPLIERS, 250 MHz, 16 GMAC/S PEAK;  
APPLE.JPG, 63% WEIGHT SPARSITY). FULL 52-LAYER BREAKDOWN  
OMITTED FOR SPACE.

| Layer                               | MACs (M) | Theo. FPS | Theo. Util | RTL FPS | RTL Util |
|-------------------------------------|----------|-----------|------------|---------|----------|
| conv1 (stem, $3 \times 3/s2$ )      | 3.02     | 5304.2    | 100%       | 2260    | 54.4%    |
| conv2 (pointwise, $1 \times 1$ )    | 17.32    | 923.7     | 100%       | 719     | 92.5%    |
| conv3 (depthwise, $3 \times 3/s1$ ) | 0.92     | 17472.4   | 100%       | 2665    | 20.3%    |
| conv4 (head, $1 \times 1$ )         | 19.08    | 838.7     | 100%       | 723     | 89.4%    |
| Network (52 layers)                 | 203.15   | 78.8      | 100%       | 56.1    | 78.9%    |

Network fps reaches 71% of this array’s theoretical peak (56.1 of 78.8 fps) – higher than AlexNet’s 61%, even though MobileNet’s per-layer multUtil spread is far wider, because the heavy pointwise layers (conv2 and conv4 in Table VII) that dominate total MAC count already run at 89-93% utilization; the low-multUtil depthwise layers are cheap enough in absolute MACs that they barely move the network average.

Our array (32 multipliers, 250 MHz) is far smaller than the original paper’s reported GoSPA/GoSPA-R configuration (128 multipliers, one per PE, at 500 MHz). To compare fairly, Table VIII scales the paper’s reported throughput linearly by the peak-MAC-rate ratio between the two array sizes/clocks ( $\frac{32 \times 250}{128 \times 500} = 0.125 \times$ ) and lines that up against our own RTL-measured fps at that same, smaller size.

TABLE VIII

OUR GoSPA VS. THE ORIGINAL PAPER, SCALED TO OUR ARRAY SIZE.

| Config                | Mults | $N_{PE}$ | Freq    | AlexNet fps | MobileNet fps |
|-----------------------|-------|----------|---------|-------------|---------------|
| Paper (GoSPA/GOSPA-R) | 128   | 32       | 500 MHz | 460.3       | 1868          |
| Paper, scaled to ours | 32    | 8        | 250 MHz | 57.5        | 233.5         |
| Ours, AlexNet         | 32    | 8        | 250 MHz | 49.6        | x             |
| Ours, MobileNet       | 64    | 16       | 250 MHz | x           | 56.1          |

At matched array size and clock, we reach 86% of the paper’s scaled AlexNet throughput (49.6 of 57.5 fps), but only 24% on MobileNet (56.1 of 233.5 fps). We had to scale our MobileNet instance to 64 multipliers because, at 32 multipliers, we could barely reach 27 fps, given a theoretical maximum of around 40 fps. Our approach for MobileNet was to optimize for area so we could pack in more multipliers compared to AlexNet; therefore running at 32 multipliers would not be representative. Overall, the significant discrepancy between our results and the paper’s for MobileNetV2 suggests that they had a different setup, possibly with several GoSPA instances instead of one. Self-comparison of our MobileNetV2 results shows the architecture does in fact work as expected regardless.

Taken together with the area numbers of Section VI-C, these results locate the remaining headroom in area rather than in multiplier efficiency: at 61% of theoretical peak fps the AlexNet configuration occupies only 31% of the device, so the fabric has room to grow  $N_{PE}$  well past 8, which scales the theoretical peak fps directly and is the most direct lever to raise measured fps further on the same device.

## VII. CONCLUSION AND FUTURE WORK

This work reproduced GoSPA’s sparse convolution accelerator end to end, implementing the APU and PE array in SystemVerilog alongside a Python functional model and an analytical performance model. Two dataflow interpretations, V1 and V2, were compared to resolve ambiguity in the paper’s multi-lane PE mapping; the analytical model showed V2’s batched activation routing was needed to clear a router-starvation bottleneck, and this decision carried forward into GoSPA V3, our final, delivered architecture (Section III-E). Run end to end on pruned AlexNet and quantized MobileNetV2 on the Kria KV260, V3 closes timing at 274.1 MHz and 250.1 MHz, reaching 61% and 71% of theoretical peak throughput. GoSPA is thus fully implemented and working, confirming that on-the-fly intersection avoids explicit intersection’s drawbacks and that realizing this in a multi-lane array requires deliberately choosing where parallelism lives.

With the architecture complete, the main opportunity left is clock frequency: both instances close timing with thin positive slack, still roughly  $2 \times$  below the original paper’s 500 MHz, so further pipelining and retiming of the APU/PE critical paths is the most direct lever to raise throughput. Secondary directions include closing the remaining utilization gap (wider on MobileNet), a per-layer-configured ( $N_{PE}, M, B$ ) design using the model’s minimum batch width  $B^*$ , and a unified V3 elaboration covering both networks. The implementation is available in our repository.

## REFERENCES

- [1] C. Deng, Y. Sui, S. Liao, X. Qian, and B. Yuan, “GoSPA: An Energy-Efficient High-Performance Globally Optimized Sparse Convolutional Neural Network Accelerator,” in *Proc. 48th Annu. Int. Symp. Computer Architecture (ISCA)*, 2021.
- [2] A. Gondimalla, N. Chesnut, M. Thottethodi, and T. N. Vijaykumar, “SparTen: A Sparse Tensor Accelerator for Convolutional Neural Networks,” in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2019, pp. 151–165.
- [3] K. Hegde, H. Asghari-Moghaddam, M. Pellauer, N. Crago, A. Jaleel, E. Solomonik, J. Emer, and C. W. Fletcher, “Extensor: An Accelerator for Sparse Tensor Algebra,” in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2019, pp. 319–333.